Observaciones – Laboratorio 9: Colas de prioridad

Grupo G05

Integrantes

Nombre completo	Código	Correo Uniandes
Sebastian David Moreno Bustos	202616523	sd.morenob1@uniandes.edu.co
Miguel Ángel Colmenares Rodríguez	202616074	m.colmenaresr2@uniandes.edu.co
Marvin Charris	202615633	m.charrism@uniandes.edu.co

Preguntas de observación

a) ¿Qué estructura de datos subyacente se utiliza comúnmente para implementar una cola de prioridad en Python?

Respuesta:

Se usa un heap (montículo) binario. Es un árbol binario completo que se guarda en un arreglo, y en él cada padre tiene igual o más prioridad que sus hijos. Así, el elemento más importante siempre queda en la raíz. En el laboratorio lo implementamos sobre un array_list: para un nodo en la posición i, su padre está en i // 2 y sus hijos en 2i y 2i + 1.

b) ¿Cuál es la diferencia entre una cola FIFO tradicional y una cola de prioridad?

Respuesta:

En una cola FIFO los elementos salen en el mismo orden en que entraron: el primero que llega es el primero que se atiende. En una cola de prioridad cada elemento tiene una prioridad, y siempre se atiende primero el más prioritario, aunque haya llegado de último. Por ejemplo, en el laboratorio las rutas de bus no salen en el orden del archivo, sino según la hora del primer bus.

c) ¿Qué módulo proporciona Python para trabajar fácilmente con colas de prioridad?

Respuesta:

El módulo heapq, que viene incluido en Python. Permite usar una lista normal como heap con funciones como heappush (insertar) y heappop (sacar el menor). También existe queue.PriorityQueue, que funciona sobre heapq. En el curso usamos nuestra propia implementación para entender cómo funciona por dentro.

d) ¿Qué ventajas tiene el uso de una cola de prioridad sobre una lista ordenada manualmente?

Respuesta:

La principal ventaja es la velocidad. En una lista ordenada, cada vez que se inserta un elemento hay que buscar su lugar y correr los demás, lo que cuesta O(n). En un heap, insertar y sacar elementos cuesta solo O(log n). Además, la cola de prioridad se encarga sola de mantener el orden, así que no hay que reordenar a mano ni arriesgarse a cometer errores.

e) Si dos elementos tienen la misma prioridad, ¿cómo decide la cola cuál atender primero?

Respuesta:

El heap no garantiza ningún orden entre elementos con la misma prioridad. Cuál sale primero depende de la posición en que haya quedado cada uno dentro del árbol, no necesariamente de cuál llegó antes. Si se quiere respetar el orden de llegada, se puede agregar un segundo criterio, como un contador que aumenta con cada inserción, para desempatar.

f) ¿Qué situaciones del mundo real se pueden modelar con colas de prioridad? Mencione al menos dos.

Respuesta:

• Urgencias de un hospital: los pacientes se atienden según su gravedad y no según su orden de llegada.

• Horarios de transporte: como en este laboratorio, se ordenan las rutas de bus según la hora en que sale el primer bus.

• Sistemas operativos: el computador ejecuta primero los procesos más importantes.

g) En un sistema de atención médica, ¿cómo se puede usar una cola de prioridad para organizar a los pacientes?

Respuesta:

Cuando llega un paciente, se le asigna una prioridad según su gravedad (por ejemplo, de 1 = muy grave a 5 = leve) y se inserta en la cola. Cada vez que un médico queda libre, se saca de la cola al paciente más grave. Si dos pacientes tienen la misma gravedad, se puede atender primero al que lleva más tiempo esperando. Y si un paciente empeora, se puede actualizar su prioridad para que lo atiendan antes.

h) ¿Cómo afectaría al rendimiento usar una lista simple en lugar de una estructura especializada como heap_pq para manejar prioridades?

Respuesta:

El programa sería más lento. Con una lista simple, para encontrar el elemento más prioritario hay que revisar todos los elementos, lo que cuesta O(n) cada vez. Con un heap, esa misma operación cuesta O(log n). Con pocos datos la diferencia casi no se nota, pero con miles de datos, como las paradas de bus del laboratorio, el heap es mucho más eficiente.

i) ¿Qué complejidad temporal tiene la inserción y eliminación/atención en una cola de prioridad basada en heap?

Respuesta:

Ambas operaciones son O(log n). Al insertar, el elemento se pone al final y va subiendo (swim) hasta su lugar. Al eliminar, se saca la raíz, se pone el último elemento arriba y se hace bajar (sink). En los dos casos el elemento recorre como máximo la altura del árbol, que es log n. Consultar el primero, sin sacarlo, es O(1).